

lab01

March 11, 2026

1 Introduction to Python

1.1 Interactive mode, lists, functions

```
[68]: 2+3  
print(2+3)  
a=2  
b=3  
a+b
```

5

[68]: 5

```
[69]: 2+3  
print(2+3)
```

5

```
[70]: a, b = 2, 3  
print(a+b)
```

5

```
[71]: l = [1, 3, [0, 1], 'c', lambda x: x*x]  
l
```

[71]: [1, 3, [0, 1], 'c', <function __main__.<lambda>(x)>]

```
[72]: l[0]  
len(l), l[4], l[len(l) - 1], 3*[0,1,2]  
#l[len(l)] # IndexError: list index out of range
```

[72]: (5,
 <function __main__.<lambda>(x)>,
 <function __main__.<lambda>(x)>,
 [0, 1, 2, 0, 1, 2, 0, 1, 2])

```
[73]: 1[-1], 1[-2], 1[-3], 1[-4], 1[-5]
      #l[-6] #IndexError: list index out of range
```

```
[73]: (<function __main__.<lambda>(x)>, 'c', [0, 1], 3, 1)
```

```
[74]: 1[2:4], 1[2:], 1[:3], 1[:]
```

```
[74]: ([[0, 1], 'c'],
      [[0, 1], 'c', <function __main__.<lambda>(x)>],
      [1, 3, [0, 1]],
      [1, 3, [0, 1], 'c', <function __main__.<lambda>(x)>])
```

```
[75]: m = list(range(10))
      m, m[::2], m[1:8:2], m[::-1]
```

```
[75]: ([0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
      [0, 2, 4, 6, 8],
      [1, 3, 5, 7],
      [9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
```

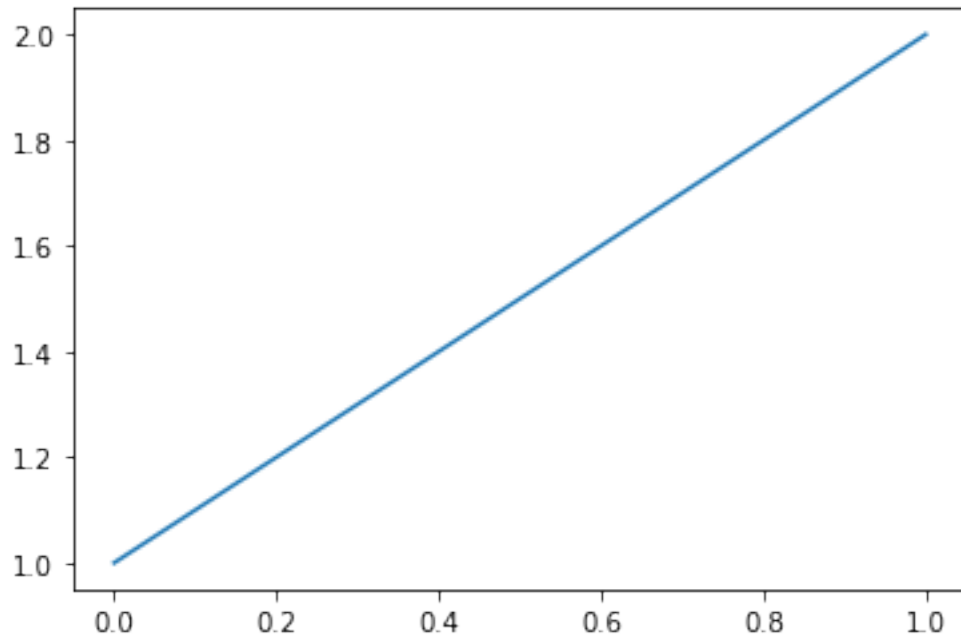
```
[76]: def f(x):
      return 2*x
      f, f(m[3]), f(m) # integer multiple of a list = tiling
```

```
[76]: (<function __main__.f(x)>,
      6,
      [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
[77]: import numpy as np
      f(np.array(m)) # multiplication of a numeric array = elementwise operations
```

```
[77]: array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
```

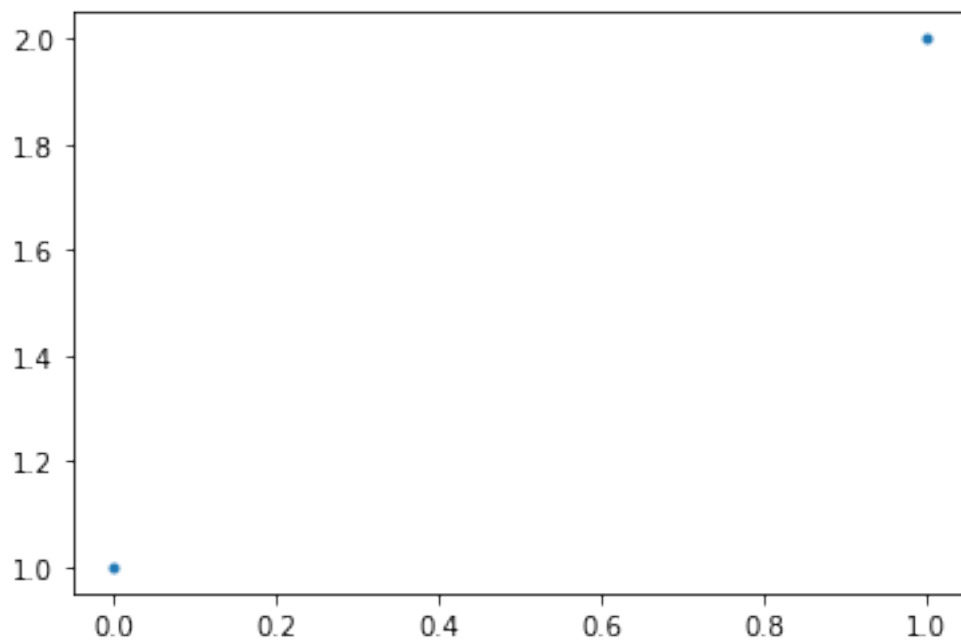
```
[78]: import matplotlib.pyplot as plt
      x = [0, 1]
      y = [1, 2]
      x, y = [0, 1], [1, 2]
      plt.plot(x, y)
      plt.show() # to be used in scripts (not in Jupyterlab)
      plt.ion() # to be used in terminal for interactive mode (not in Jupyterlab)
```



[78]: <matplotlib.pyplot._IonContext at 0x7f88499c61f0>

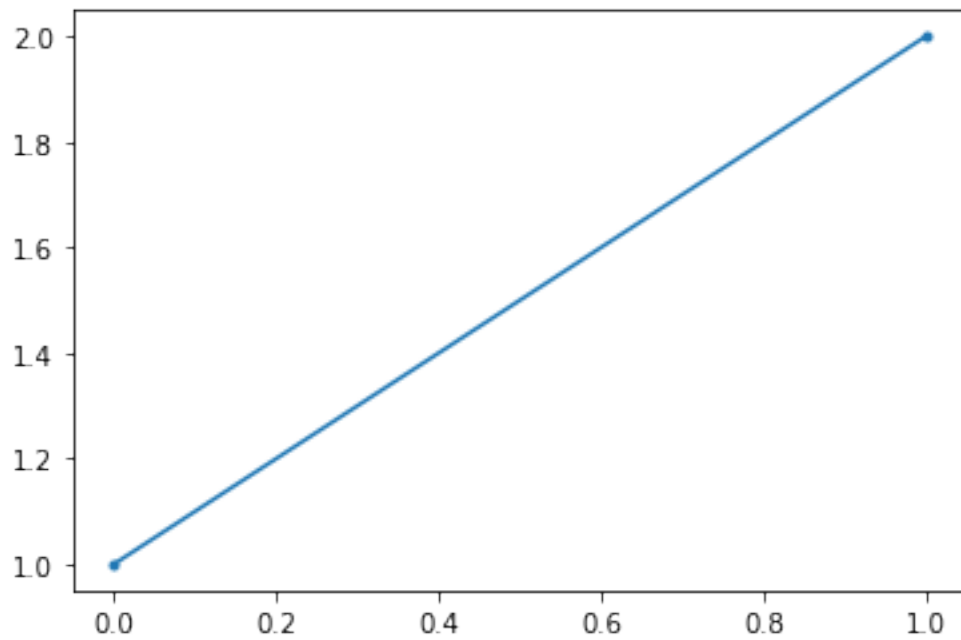
```
[79]: plt.plot(x, y, '.')
```

[79]: [<matplotlib.lines.Line2D at 0x7f8849bf61f0>]



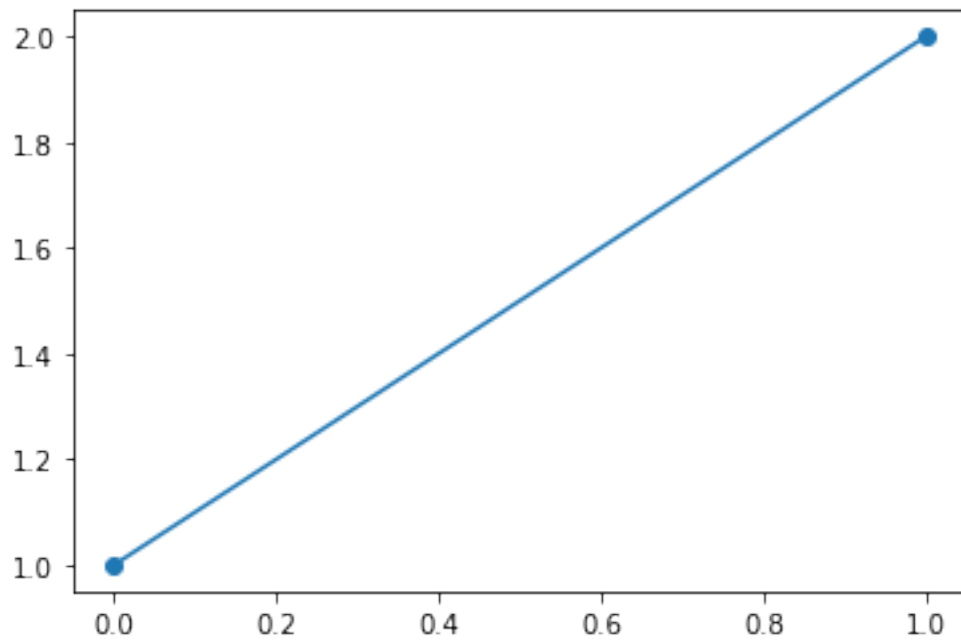
```
[80]: plt.plot(x, y, '.-')
```

```
[80]: [<matplotlib.lines.Line2D at 0x7f8849ccb820>]
```



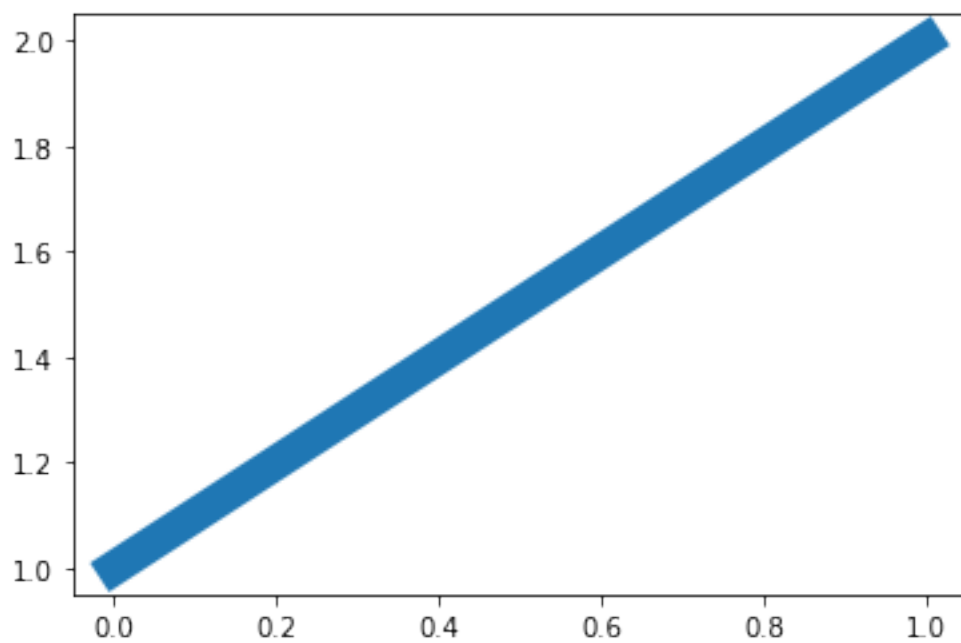
```
[81]: plt.plot(x, y, 'o-')
```

```
[81]: [<matplotlib.lines.Line2D at 0x7f884916df40>]
```



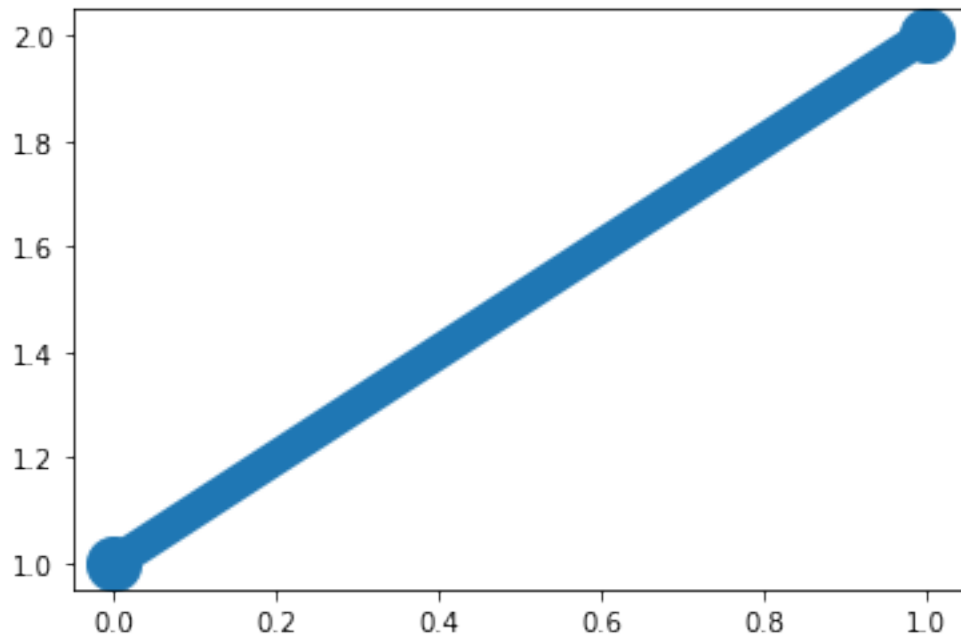
```
[82]: plt.plot(x, y, 'o-', linewidth=13)
```

```
[82]: [<matplotlib.lines.Line2D at 0x7f884915b520>]
```

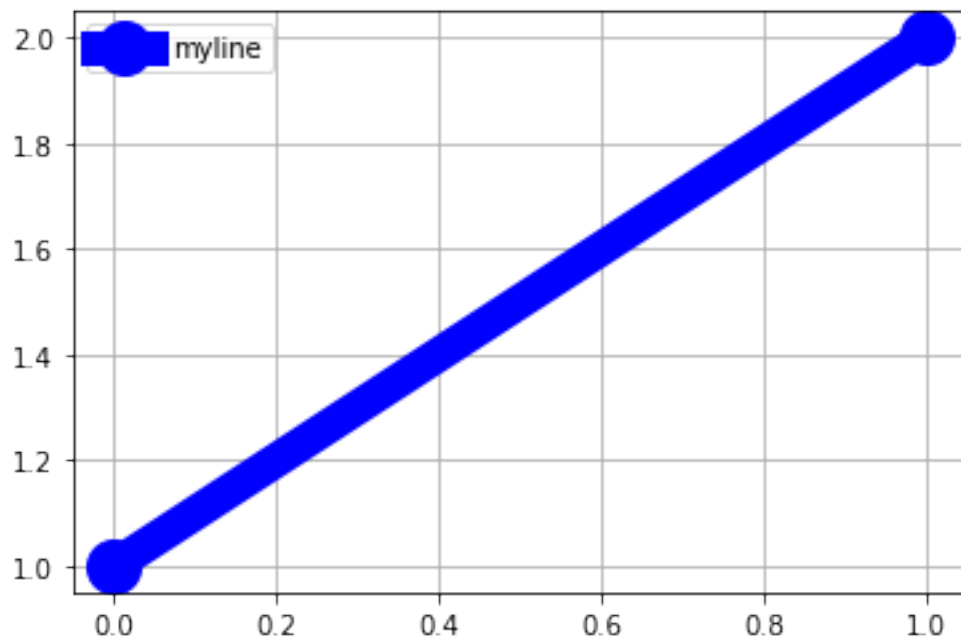


```
[83]: plt.plot(x, y, 'o-', linewidth=13, markersize=20)
```

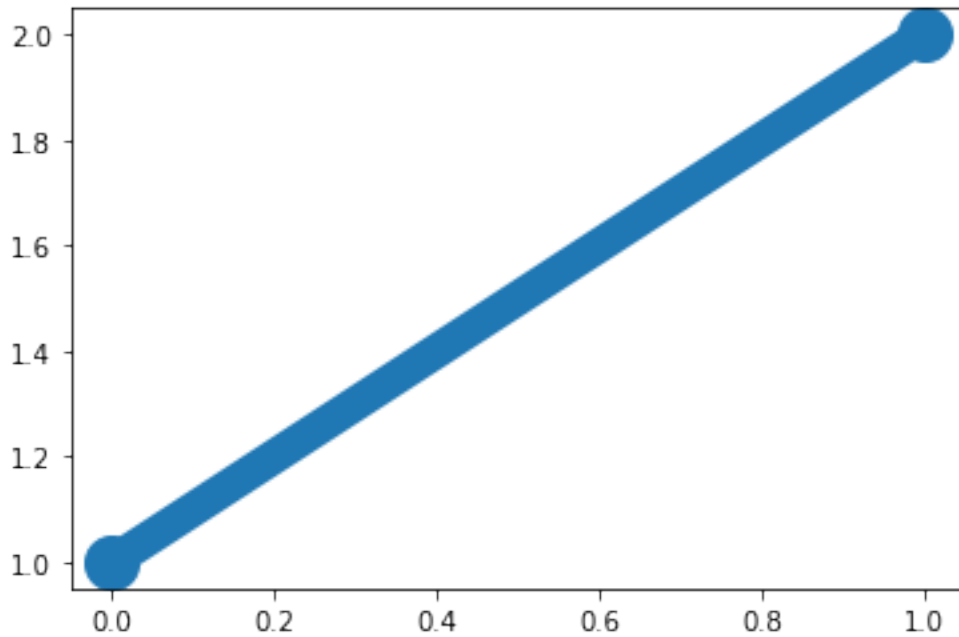
```
[83]: [<matplotlib.lines.Line2D at 0x7f88490c8040>]
```



```
[84]: plt.plot(x, y, 'bo-', linewidth=13, markersize=20, label='myline')  
plt.legend()  
plt.grid() # or plt.grid(True)
```



```
[85]: plt.plot(x, y, 'o-', linewidth=13, markersize=20)
plt.grid()
plt.grid(False)
```



```
[86]: help(plt.plot)
```

Help on function plot in module matplotlib.pyplot:

```
plot(*args, scalex=True, scaley=True, data=None, **kwargs)
    Plot y versus x as lines and/or markers.
```

Call signatures::

```
plot([x], y, [fmt], *, data=None, **kwargs)
plot([x], y, [fmt], [x2], y2, [fmt2], ..., **kwargs)
```

The coordinates of the points or line nodes are given by **x**, **y**.

The optional parameter **fmt** is a convenient way for defining basic formatting like color, marker and linestyle. It's a shortcut string notation described in the **Notes** section below.

```
>>> plot(x, y)          # plot x and y using default line style and color
>>> plot(x, y, 'bo')    # plot x and y using blue circle markers
```

```
>>> plot(y)           # plot y using x as index array 0..N-1
>>> plot(y, 'r+')      # ditto, but with red plusses
```

You can use ``.Line2D`` properties as keyword arguments for more control on the appearance. Line properties and `*fmt*` can be mixed. The following two calls yield identical results:

```
>>> plot(x, y, 'go--', linewidth=2, markersize=12)
>>> plot(x, y, color='green', marker='o', linestyle='dashed',
...      linewidth=2, markersize=12)
```

When conflicting with `*fmt*`, keyword arguments take precedence.

****Plotting labelled data****

There's a convenient way for plotting objects with labelled data (i.e. data that can be accessed by index ```obj['y']```). Instead of giving the data in `*x*` and `*y*`, you can provide the object in the `*data*` parameter and just give the labels for `*x*` and `*y*::`

```
>>> plot('xlabel', 'ylabel', data=obj)
```

All indexable objects are supported. This could e.g. be a ``dict``, a ``pandas.DataFrame`` or a structured numpy array.

****Plotting multiple sets of data****

There are various ways to plot multiple sets of data.

- The most straight forward way is just to call ``plot`` multiple times.
Example:

```
>>> plot(x1, y1, 'bo')
>>> plot(x2, y2, 'go')
```

- If `*x*` and/or `*y*` are 2D arrays a separate data set will be drawn for every column. If both `*x*` and `*y*` are 2D, they must have the same shape. If only one of them is 2D with shape `(N, m)` the other must have length `N` and will be used for every data set `m`.

Example:

```
>>> x = [1, 2, 3]
>>> y = np.array([[1, 2], [3, 4], [5, 6]])
>>> plot(x, y)
```

is equivalent to:

```
>>> for col in range(y.shape[1]):  
...     plot(x, y[:, col])
```

- The third way is to specify multiple sets of `*[x]*`, `*y*`, `*[fmt]*` groups::

```
>>> plot(x1, y1, 'g^', x2, y2, 'g-')
```

In this case, any additional keyword argument applies to all datasets. Also this syntax cannot be combined with the `*data*` parameter.

By default, each line is assigned a different style specified by a 'style cycle'. The `*fmt*` and line property parameters are only necessary if you want explicit deviations from these defaults. Alternatively, you can also change the style cycle using `:rc:`axes.prop_cycle``.

Parameters

`x, y` : array-like or scalar

The horizontal / vertical coordinates of the data points.
`*x*` values are optional and default to ``range(len(y))``.

Commonly, these parameters are 1D arrays.

They can also be scalars, or two-dimensional (in that case, the columns represent separate data sets).

These arguments cannot be passed as keywords.

`fmt` : str, optional

A format string, e.g. 'ro' for red circles. See the `*Notes*` section for a full description of the format strings.

Format strings are just an abbreviation for quickly setting basic line properties. All of these and more can also be controlled by keyword arguments.

This argument cannot be passed as keyword.

`data` : indexable object, optional

An object with labelled data. If given, provide the label names to plot in `*x*` and `*y*`.

.. note::

Technically there's a slight ambiguity in calls where the second label is a valid `*fmt*`. ```plot('n', 'o', data=obj)``` could be ```plt(x, y)``` or ```plt(y, fmt)```. In such cases, the former interpretation is chosen, but a warning is issued. You may suppress the warning by adding an empty format string ```plot('n', 'o', '', data=obj)```.

Returns

list of ``.Line2D``

A list of lines representing the plotted data.

Other Parameters

`scalex, scaley` : bool, default: True

These parameters determine if the view limits are adapted to the data limits. The values are passed on to ``.autoscale_view``.

****kwargs** : ``.Line2D`` properties, optional

kwargs are used to specify properties like a line label (for auto legends), linewidth, antialiasing, marker face color.

Example::

```
>>> plot([1, 2, 3], [1, 2, 3], 'go-', label='line 1', linewidth=2)
>>> plot([1, 2, 3], [1, 4, 9], 'rs', label='line 2')
```

If you specify multiple lines with one plot call, the kwargs apply to all those lines. In case the label object is iterable, each element is used as labels for each set of data.

Here is a list of available ``.Line2D`` properties:

Properties:

`agg_filter`: a filter function, which takes a (m, n, 3) float array and a dpi value, and returns a (m, n, 3) array

`alpha`: scalar or None

`animated`: bool

`antialiased` or `aa`: bool

`clip_box`: ``.Bbox``

`clip_on`: bool

`clip_path`: Patch or (Path, Transform) or None

`color` or `c`: color

`contains`: unknown

`dash_capstyle`: ``.CapStyle`` or {'butt', 'projecting', 'round'}

`dash_joinstyle`: ``.JoinStyle`` or {'miter', 'round', 'bevel'}

`dashes`: sequence of floats (on/off ink in points) or (None, None)

`data`: (2, N) array or two 1D arrays

```

    drawstyle or ds: {'default', 'steps', 'steps-pre', 'steps-mid', 'steps-
post'}, default: 'default'
    figure: `.Figure`
    fillstyle: {'full', 'left', 'right', 'bottom', 'top', 'none'}
    gid: str
    in_layout: bool
    label: object
    linestyle or ls: {'-', '--', '-.', ':', '', (offset, on-off-seq), ...}
    linewidth or lw: float
    marker: marker style string, `~.path.Path` or `~.markers.MarkerStyle`
    markeredgecolor or mec: color
    markeredgewidth or mew: float
    markerfacecolor or mfc: color
    markerfacecoloralt or mfcalt: color
    markersize or ms: float
    markevery: None or int or (int, int) or slice or list[int] or float or
(float, float) or list[bool]
    path_effects: `.AbstractPathEffect`
    picker: float or callable[[Artist, Event], tuple[bool, dict]]
    pickradius: float
    rasterized: bool
    sketch_params: (scale: float, length: float, randomness: float)
    snap: bool or None
    solid_capstyle: `.CapStyle` or {'butt', 'projecting', 'round'}
    solid_joinstyle: `.JoinStyle` or {'miter', 'round', 'bevel'}
    transform: `matplotlib.transforms.Transform`
    url: str
    visible: bool
    xdata: 1D array
    ydata: 1D array
    zorder: float

```

See Also

`scatter` : XY scatter plot with markers of varying size and/or color (sometimes also called bubble chart).

Notes

****Format Strings****

A format string consists of a part for color, marker and line::

```
fmt = '[marker][line][color]'
```

Each of them is optional. If not provided, the value from the style cycle is used. Exception: If ``line`` is given, but no ``marker``, the data will be a line without markers.

Other combinations such as ``[color][marker][line]`` are also supported, but note that their parsing may be ambiguous.

****Markers****

character	description
``'.'``	point marker
``',``	pixel marker
``'o'``	circle marker
``'v'``	triangle_down marker
``'^'``	triangle_up marker
``'<'``	triangle_left marker
``'>'``	triangle_right marker
``'1'``	tri_down marker
``'2'``	tri_up marker
``'3'``	tri_left marker
``'4'``	tri_right marker
``'8'``	octagon marker
``'s'``	square marker
``'p'``	pentagon marker
``'P'``	plus (filled) marker
``'*'``	star marker
``'h'``	hexagon1 marker
``'H'``	hexagon2 marker
``'+'``	plus marker
``'x'``	x marker
``'X'``	x (filled) marker
``'D'``	diamond marker
``'d'``	thin_diamond marker
``' '``	vline marker
``'_'``	hline marker

****Line Styles****

character	description
``'-'``	solid line style
``'--'``	dashed line style
``'-.'``	dash-dot line style
``':'``	dotted line style

Example format strings::

```
'b'      # blue markers with default shape
'or'     # red circles
'-g'     # green solid line
'--'     # dashed line with default color
'^k:'    # black triangle_up markers connected by a dotted line
```

****Colors****

The supported color abbreviations are the single letter codes

character	color
``'b'``	blue
``'g'``	green
``'r'``	red
``'c'``	cyan
``'m'``	magenta
``'y'``	yellow
``'k'``	black
``'w'``	white

and the ``'CN'`` colors that index into the default property cycle.

If the color is the only part of the format string, you can additionally use any ``matplotlib.colors`` spec, e.g. full names (```'green'```) or hex strings (```'#008000'```).

```
[87]: N = 10
      impulse = [0]*N
      impulse[0] = 1
      impulse
```

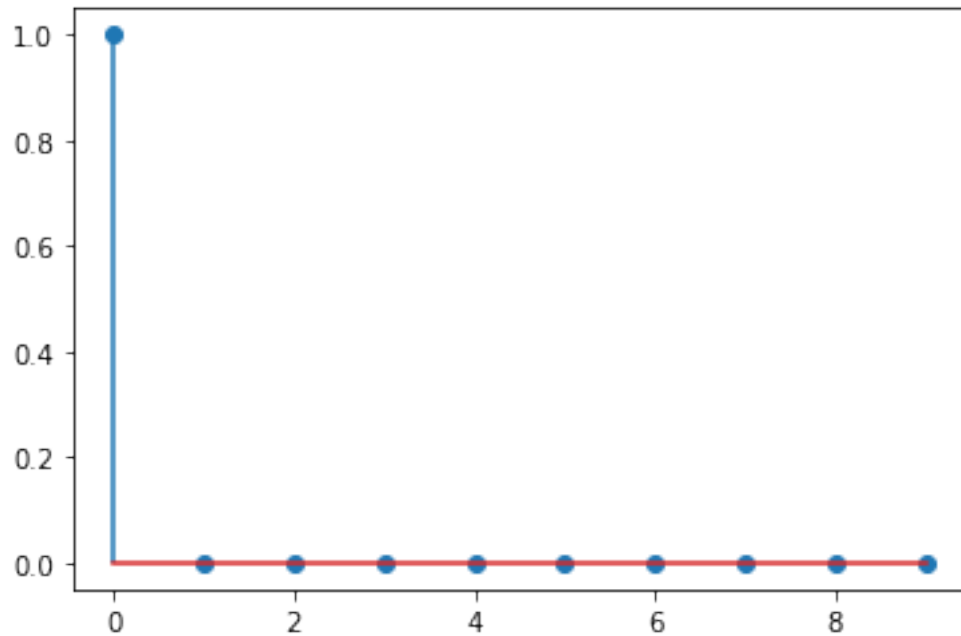
```
[87]: [1, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
[88]: np.roll(impulse, N//2)
```

```
[88]: array([0, 0, 0, 0, 0, 0, 1, 0, 0, 0])
```

```
[89]: plt.stem(impulse)
```

```
[89]: <StemContainer object of 3 artists>
```



```
[90]: n = np.arange(N) - N//2  
      n
```

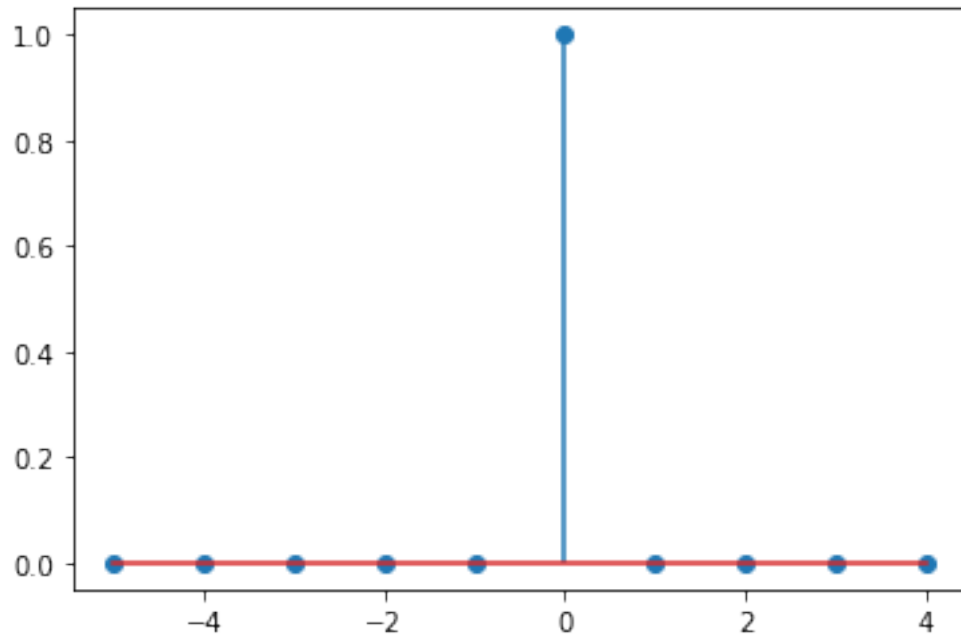
```
[90]: array([-5, -4, -3, -2, -1,  0,  1,  2,  3,  4])
```

```
[91]: impulse = np.zeros(N)  
      impulse[0] = 1  
      impulse
```

```
[91]: array([1., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

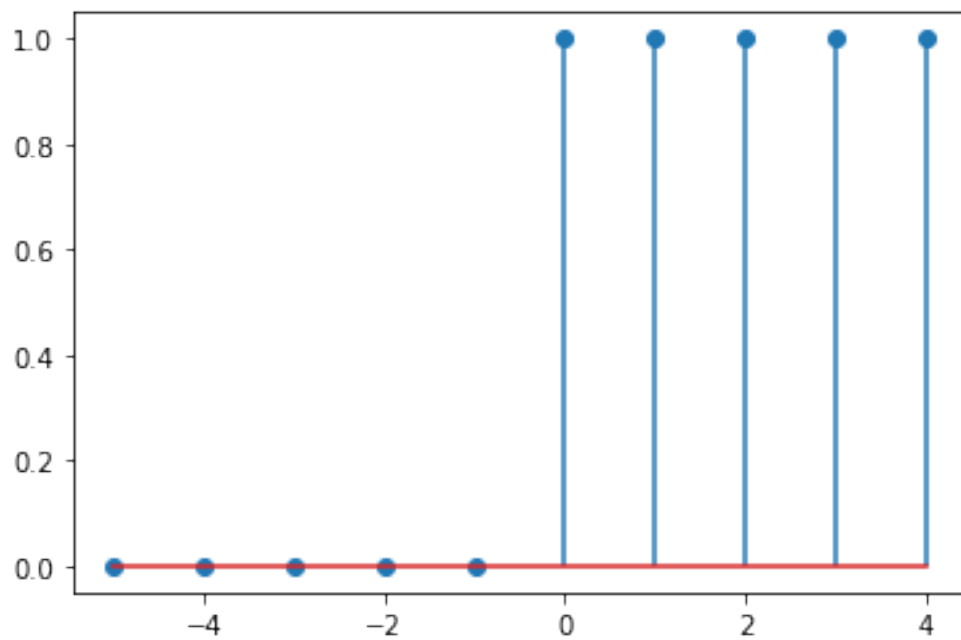
```
[92]: plt.stem(n, impulse[n])
```

```
[92]: <StemContainer object of 3 artists>
```



```
[93]: step = np.zeros(N)
      step[:N//2] = 1
      step
      plt.stem(n, step[n])
```

[93]: <StemContainer object of 3 artists>



1.2 Intro to plotting